Lista 02 - Victor Yaegashi Setti - RA 206362

Questão 1

a) Propondo o algoritmo:

Supondo que as operações de inserção, remoção e subvetorização de vetor `inserir(V, u)`, `remover(V, index)` e `subvetor(V, i, f)` são todas $O(1)$:

```
  bruxaria(A, k):
01    S = []
02    if (!Magico(A, k)):
03        return S
04
05    soma = k
06    for (i = 0; i < tamanho(A) and soma != 0; i++):
07        temp = A[i]
08        if ((i + 1 < n and !Magico(subvetor(A, i+1, n), soma)) or Magico(tem
p, soma)):
09            adiciona(S, temp)
10            soma = soma - temp
11    return S
```

b) Provando a corretude do algoritmo:

Seja a invariante do laço na linha 06 tal que:

$P(i) \Leftrightarrow$ o vetor `S` é tal que, até a posição i em A, $\sum_{j=0}^{|S|-1} S[j] + \text{soma} = \text{k}$ e `Magico(subvetor(A, i, n), soma)` é verdade.

Provando a invariante:

Antes de entrar no laço, o algoritmo termina caso `Magico(A, k)` retorne falso, indicando que não há subconjunto que satisfaça o requisito.

Caso `Magico(A, k)` retorna vazio teremos que:

`S = []` $\Rightarrow \sum_{j=0}^{|S|-1} S[j] = 0$
`soma = k`

$\Rightarrow \sum_{j=0}^{|S|-1} S[j] + soma = 0 + k = k$

Além disso, como já vimos que `Magico(A, k)` retorna verdadeiro para este caso $\Rightarrow$ A invariante vale.

Supondo uma iteração qualquer do laço tal que $P(c)$, na iteração seguinte teremos `c' = c + 1` e `temp = A[c']`, divindo em dois casos:

- Caso `Magico(subvetor(A, c + 1, n), soma - temp)` retorna verdadeiro $\Rightarrow$ `soma' = soma - temp` e `adiciona(S, x)` $\Rightarrow (\sum_{j=0}^{|S|-1} S[j])' = \sum_{j=0}^{|S|-1} S[j] + temp \Rightarrow (\sum_{j=0}^{|S|-1} S[j])' + soma' = \sum_{j=0}^{|S|-1} S[j] + temp + soma - temp = k$ (utilizando $P(c)$).
  Além disso, como temos também que `Magico(subvetor(A, c + 1, n), soma - temp)` = `Magico(subvetor(A, c', n), soma')` = verdadeiro.
  $\Rightarrow P(c+1)$ vale.

- No caso contrário teremos que `Magico(subvetor(A, c + 1, n), soma - temp)` retorna falso $\Rightarrow$ `temp` não pertence a solução $\Rightarrow$ tanto `S` quanto `soma` permanencem inalterados e valendo.
  Além disso, teremos que `Magico(subvetor(A, c + 1, n), soma)` = `Magico(subvetor(A, c', n), soma)` = verdadeiro $\Rightarrow P(c+1)$ vale.

Assim, a invariante vale $\forall i \in [0, |A|]$ no laço da linha 06.

Sabendo que a condição de parada do laço é `i = |A|`, e dada a invariante, teremos que para $P(|A|)$ `subvetor(A, |A|, |A|)` é vazio e `Magico(subvetor(A, |A|, |A|), soma)` retorna verdadeiro $\Rightarrow$ soma = 0 $\Rightarrow k = \sum_{j=0}^{|S|-1} S[j]$.

$\therefore$ Como o algoritmo sempre retorna S em P(n), é provada a corretude. ■

c) Provando a complexidade temporal:

Antes do laço são realizadas apenas operaçõs de complexidade temporal constante.
Como o único laço do algoritmo `bruxaria` é um `for` que itera de 0 até n, sendo `n = tamanho(A)`, e dentro deste laço todas as operações são $O(1)$, teremos uma complexidade $T(n) = n \cdot O(1) \Rightarrow T(n) \in O(n)$.
$\therefore$ O algoritmo `bruxaria` possui complexidade O(n) ■

Questão 2

Usando o algoritmo dado em sala `intercala` podemos construir o algoritmo:

```
merge_sort3(V, i, f):
    if (i <= f):
        return

    x = i + (f-1)/3
    y = i + 2(f-1)/3

    merge_sort3(V, i, x)
    merge_sort3(V, x+1, y)
    merge_sort3(V, y+1, f)
    intercala(V, i, x, y)
    intercala(V, i, y, f)

    return
```

Agora provando a complexidade de `merge_sort3`:
Temos pelo enunciado que x - i + 1 $\approx$ y - x $\approx$ f - y $\approx \frac{n}{3}$, sendo n o tamanho do vetor. Temos também que, conforme visto em aua, a complexidade temporal de `intercala` é $\Theta(n)$.

Com isso, temos que a complexidade temporal das duas chamadas de `intercala` será, com c e c_0 constantes:

$$c \cdot 2 \cdot \frac{n}{3} + cn = c_0 n \in \Theta(n)$$

Assim, a complexidade temporal de `merge_sort3` será representada pela expressão, com $c_1 \in \mathbb{N}$:

$$\begin{cases} T(n) = 3T(\frac{n}{2}) + nc_1, n \geq 3 \\ T(n) = c_1, n < 2 \end{cases}$$

Portanto, utilizando o Teorema Mestre dado em aula com $a = 3$ e $b = 3$ teremos que:

$$n^{\log_b a} = n^{\log_3 3} = n \in \Theta(n)$$
$$\Rightarrow T(n) \in \Theta(n \cdot logn)$$

$\therefore$ O algoritmo `merge_sort3` possui complexidade $\Theta(n \cdot logn)$ ■

Questão 3

```
majoritario(V, i, f):
  if (i > f):
      return null
  if (i == f):
      return V[i]

   n = f - i + 1
   maj_e = majoritario(V, i, n/2)
   maj_d = majoritario(V, n/2 + 1, f)
   cont_e = 0
   cont_d = 0

   for (k = i; k <= f; k++)
       if (maj_e = V[k]):
           count_e++
       if (maj_d = V[k]):
           count_d++

   if (cont_e >= n/2 + 1):
       return maj_e
   if (cont_d >= n/2 + 1):
       return maj_d
   else:
       return null
```

Questão 4

Seja:

$$X = 10a_2 + a_1$$
$$Y = 10b_2 + b_1$$

$$A = 10X + a_0$$
$$B = 10Y + b_0$$

Calculando $X \cdot Y$:

$$X \cdot Y = (10a_2 + a_1) \cdot (10b_2 + b_1)$$
$$X \cdot Y = (100a_2b_2) + 10(a_2b_1 + a_1b_2) + a_1b_1$$

Sabendo que:

$$AB = XY \cdot 100 + (Xb_0 + Ya_0) \cdot 10 + a_0b_0$$

Podemos definir:

$$M_0 = a_0b_0$$
$$M_1 = XY$$
$$M_2 = (X + a_0)(Y + b_0)$$

E com isso podemos calcular o termo $(Xb_0 + Ya_0)$, pois:

$$(Xb_0 + Ya_0) = M_2 - M_1 - M_0$$

Assim, teremos todos os termos necessários para calcular AB com apenas 7 multiplicações:

X: 1 multiplicação
Y: 1 multiplicação
Xb_0: 1 multiplicação
Ya_0: 1 multiplicação
a_0b_0: 1 multiplicação (M_0)
XY: 1 multiplicação(M_1)
$(X + a_0)(Y + b_0)$: 1 multiplicação(M_2)

$\therefore$ 7 multiplicações ■

Questao 5

```
inverte(V, i, f):
    if (i >= f):
        return
    inverte(V, i+1, f-1)
    V[i] <-> V[f]
    return
```

Provando o algoritmo por indução:
Seja P(n) <=> Dado um vetor V de tamanho n = f - i + 1, `inverte(V, i, f)` inverte V

Caso Base:

- Para n = 0:
  0 = f - i + 1
  i = f + 1
  => i $\geq$ f => retorna na linha 2
  Num vetor vazio a sua inversão é ele mesmo => P(0)

- Para n = 1:
  1 = f - i + 1
  i = f
  => i $\geq$ f => retorna na linha 2
  Num vetor com um único elemento a sua inversão é ele mesmo => P(1)

Hipótese Indutiva:
P(a) vale $\forall a \in \mathbb{N} | 0 \leq a \leq k, k \in \mathbb{N}_{\geq 1}$

Passo Indutivo:
Pela HI $k \geq 1 \Rightarrow k + 1 \geq 2$, então:
k + 1 = f - i + 1
f = i + k $\geq$ i + 1
=> Pula pela linha 2, sem retornar

Seja n' o tamanho do vetor que será passado para `inverte(V, i+1, f-1)` teremos que:
n' = (f-1) - (i+1) + 1
n' = f - i - 1
Como k = f - i temos:
n' = k - 1

Como k+1 $\geq$ 2 => k-1 $\geq$ 0. Não só isso, mas k-1 $\leq$ k. => Pela HI, P(k-1) vale.

Com isso, temos que o vetor estará invertido corretamente de i+1 a f-1 e, uma vez que o centro deste subvetor é o mesmo que o do vetor de i a f, teremos que para terminar a inversão basta trocar V[i] e V[f], o que é feito na linha 4 do pseudocódigo.

=> P(k+1)

$\therefore$ Pelo PIC, $\Rightarrow P(n) \forall n \in \mathbb{N}$

Além disso, como a cada chamada o tamanho do subvetor diminiu, sabemos que o algoritmo acaba. Assim, `inverte(V, i, f)` termina e inverte o vetor corretamente. ■